BorrowBox

Full Project Documentation

Campus-wide borrowing and lending management system

Requirements, architecture, database design, APIs, screens, business rules, security, testing, deployment and sprint plan.

Contents

1. Project Overview
2. Scope
3. User Roles
4. Functional Requirements
5. Non-Functional Requirements
6. Core Business Rules
7. Borrow Request Lifecycle
8. System Architecture
9. Database Design
10. REST API Specification
11. Scheduled Jobs
12. User Interface Specification
13. Security and Privacy
14. Suggested Project Structure
15. Testing Strategy
16. Deployment and DevOps
17. Development Plan
18. Risks and Mitigations
19. Future Enhancements
20. Glossary
21. Quick-Start Build Checklist

1. Project Overview

Problem. Students often need an item (calculator, lab coat, book, sports gear) for a short time, while others have the same item sitting unused. There is no trusted, central place on campus to connect them.

Solution. BorrowBox is a web platform where campus members list items they are willing to lend, search for items they need, request them for a period, and track the full lifecycle (request, approval, handover, return). Admins monitor users, reports and overdue items.

Goals

- Reduce unnecessary purchases and waste.
- Improve utilisation of campus resources.
- Build trust through ratings, history and transparency.
- Keep the experience simple enough to use between classes, on a phone.

Success metrics

- Number of items listed and active users per month.
- Percentage of requests that are approved and completed.
- Overdue rate (target below 10%).
- Average rating of lenders and borrowers.

2. Scope

In scope (MVP): registration and login with campus email, item listing, search and filter, borrow requests and approvals, availability tracking, borrowing history, due-date reminders, ratings and feedback, item and user reporting, admin dashboard.

Out of scope (MVP): payments or deposits, in-app chat, native mobile apps, delivery logistics, insurance. See Section 19 for future enhancements.

3. User Roles

Role	Description	Key permissions
Guest	Not logged in	View landing page, register, log in
Member (Student/Staff)	Verified campus user	List items, browse, request, approve or reject requests on own items, rate, report
Admin	Platform moderator	Everything a member can, plus manage users, items, reports, view analytics, send announcements

A single member can act as both **lender** (owner) and **borrower** depending on the transaction.

4. Functional Requirements

4.1 Authentication and Profile

- **FR-1** Register with name, campus email, password, department and phone (optional).
- **FR-2** Email verification link; only verified users can borrow or lend. Restrict to the campus email domain.
- **FR-3** Login, logout, forgot and reset password.
- **FR-4** Edit profile (name, photo, department, contact preference).

- **FR-5** Public profile shows rating, items listed and completed loans.

4.2 Item Management (Lender)

- **FR-6** Create an item: name, category, description, condition, images (up to 5), maximum loan duration (days), pickup location, optional deposit note.
- **FR-7** Edit, pause (temporarily unavailable) or delete an item. Items with active loans cannot be deleted.
- **FR-8** View all own items with status and upcoming requests.

4.3 Discovery (Borrower)

- **FR-9** Search by keyword (name and description).
- **FR-10** Filter by category, condition, availability, location, maximum duration, owner rating.
- **FR-11** Sort by newest, rating, or soonest available.
- **FR-12** Item detail page with gallery, owner info, availability calendar and reviews.

4.4 Request Management

- **FR-13** Borrower submits a request with start date, end date and optional message.
- **FR-14** System blocks requests that overlap an approved loan or exceed the maximum duration.
- **FR-15** Owner can approve or reject (with reason). Owner is notified of new requests.
- **FR-16** Borrower can cancel a request before handover.
- **FR-17** Owner marks item as **Handed Over**; borrower confirms receipt.
- **FR-18** Owner or borrower marks item as **Returned**; the other side confirms. Owner can note the return condition.
- **FR-19** Requests auto-expire if the owner does not respond within 48 hours (configurable).

4.5 Availability Tracking

- **FR-20** Item status is derived from loans: Available, Reserved (approved, future), On Loan, Paused.
- **FR-21** Availability calendar shows blocked date ranges.

4.6 Notifications and Reminders

- **FR-22** In-app and email notifications for: new request, approval, rejection, handover, return, review received.
- **FR-23** Due-date reminders at 1 day before, on the due date, and daily when overdue.
- **FR-24** Notification centre with read and unread state.

4.7 Ratings and Feedback

- **FR-25** After a completed loan, both parties can rate (1 to 5) and comment once.
- **FR-26** Average ratings are shown on user profiles and item pages.

4.8 Reporting and Moderation

- **FR-27** Members can report an item or user with a reason and description.
- **FR-28** Admin can review reports and warn, suspend or ban users, or remove items.

4.9 Admin Dashboard

- **FR-29** Overview cards: total users, active items, active loans, overdue loans, open reports.
- **FR-30** Tables with search and filters for users, items, loans and reports.
- **FR-31** Overdue list with a one-click reminder action.
- **FR-32** Manage categories and system settings (max loan days, request expiry, reminder timing).
- **FR-33** Charts: loans per week, top categories, top lenders.

5. Non-Functional Requirements

Area	Requirement
Performance	Pages load in under 2 s on a typical campus network; search responds in under 500 ms for 50k items
Security	HTTPS, hashed passwords, RBAC, input validation, rate limiting (see Section 13)
Availability	99% uptime target during term time
Usability	Mobile-first responsive UI, WCAG 2.1 AA colour contrast, keyboard accessible
Scalability	Stateless API so it can scale horizontally; DB indexes on search and date columns
Maintainability	Modular code, linting, automated tests, API documented with OpenAPI
Privacy	Show phone and email only after a request is approved; allow account deletion

6. Core Business Rules

1. A user cannot borrow their own item.
2. A loan cannot exceed the item's maximum duration (default 14 days, admin-configurable).
3. Two approved loans for the same item cannot overlap in dates.
4. A user may have at most **3 active loans** and **5 pending requests** at once (configurable).
5. A user with an overdue loan cannot make new requests until it is returned.
6. Only one review per party per loan.
7. Three or more upheld reports lead to automatic suspension pending admin review.
8. Extension: borrower may request one extension; the owner must approve, and it must not collide with another approved loan.

7. Borrow Request Lifecycle

Each loan moves through the states below. All transitions are enforced in a single LoanService.

From state	Event	To state
(start)	Borrower submits request	Pending
Pending	Owner approves	Approved
Pending	Owner rejects	Rejected
Pending	Borrower cancels	Cancelled
Pending	No response in 48h	Expired
Approved	Either party cancels	Cancelled
Approved	Handover confirmed	Active
Active	Return confirmed	Returned

From state	Event	To state
Active	Due date passed	Overdue
Overdue	Return confirmed	Returned
Returned	Reviews submitted or 7 days pass	Completed

Terminal states: Rejected, Cancelled, Expired, Completed.

8. System Architecture

Style: three-tier web application with a REST API.

```
[ Browser / Mobile Web ]
         |  HTTPS
[ React frontend (SPA) ]
         |  JSON over REST
[ Node.js + Express API ] -- [ Auth middleware / RBAC ]
         |            |                |
   [ PostgreSQL ] [ Object storage ] [ Email service ]
                  (item images)      (SMTP / SendGrid)
         |
[ Scheduler (node-cron): reminders, expiry, overdue marking ]
```

Recommended Technology Stack

Layer	Choice	Reason
Frontend	React + Vite, React Router, Tailwind CSS, React Query	Fast to build, strong ecosystem
Backend	Node.js, Express, TypeScript	One language across the stack
ORM	Prisma	Typed schema and migrations
Database	PostgreSQL	Relational integrity, date-range queries, full-text search
Auth	JWT (access + refresh) with bcrypt	Stateless and standard
File storage	Cloudinary or AWS S3	Offloads image handling
Email	Nodemailer with SMTP, or SendGrid	Verification and reminders
Scheduler	node-cron (or BullMQ for scale)	Reminder jobs
Validation	Zod	Shared request schemas
Testing	Jest, Supertest, Playwright	Unit, API and end-to-end
Deployment	Docker, Render/Railway/AWS, GitHub Actions	Simple CI/CD

Alternative: a MERN stack with MongoDB, Django + DRF, or Spring Boot can substitute with the same design. A relational database is recommended because of the strong date-overlap and integrity rules.

9. Database Design

9.1 Entity Relationships

Entity	Relationship	Entity
User	owns (1 to many)	Item
User	borrows (1 to many)	Loan
Item	is borrowed in (1 to many)	Loan
Category	classifies (1 to many)	Item
Item	has (1 to many)	ItemImage
Loan	receives (1 to many)	Review
User	writes (1 to many)	Review

Entity	Relationship	Entity
User	receives (1 to many)	Notification
User	files (1 to many)	Report
Item	is subject of (1 to many)	Report

9.2 Tables

users

Column	Type	Notes
id	UUID PK	
name	varchar(100)	
email	varchar(150) unique	Campus domain
password_hash	varchar	bcrypt
role	enum(MEMBER, ADMIN)	default MEMBER
status	enum(ACTIVE, SUSPENDED, BANNED)	
department	varchar(100)	
phone	varchar(20)	nullable
avatar_url	text	nullable
email_verified	boolean	default false
rating_avg	numeric(3,2)	denormalised
rating_count	int	
created_at, updated_at	timestamptz	

categories: id, name (unique), icon, is_active.

items

Column	Type	Notes
id	UUID PK	
owner_id	FK users	
category_id	FK categories	
name	varchar(150)	
description	text	
condition	enum(NEW, GOOD, FAIR, WORN)	
max_loan_days	int	default 14
pickup_location	varchar(200)	
is_paused	boolean	default false
is_removed	boolean	admin takedown
rating_avg	numeric(3,2)	
created_at, updated_at	timestamptz	

Indexes: GIN full-text index on name and description, plus indexes on category_id and owner_id.

item_images: id, item_id FK, url, position.

loans

Column	Type	Notes
id	UUID PK	

Column	Type	Notes
item_id	FK items	
borrower_id	FK users	
status	enum(PENDING, APPROVED, REJECTED, CANCELLED, EXPIRED, ACTIVE, OVERDUE, RETURNED, COMPLETED)	
start_date, end_date	date	requested period
message	text	borrower note
rejection_reason	text	nullable
handed_over_at	timestamptz	nullable
returned_at	timestamptz	nullable
return_condition_note	text	nullable
extension_requested_until	date	nullable
created_at, updated_at	timestamptz	

Constraint: use a PostgreSQL exclusion constraint on (item_id, daterange(start_date, end_date, '[]')) for statuses APPROVED, ACTIVE and OVERDUE so overlapping bookings are impossible even under concurrent requests.

reviews: id, loan_id FK, reviewer_id FK, reviewee_id FK, rating (1 to 5), comment, created_at. Unique on (loan_id, reviewer_id).

notifications: id, user_id FK, type, title, body, link, is_read, created_at.

reports: id, reporter_id FK, target_type (ITEM, USER), target_id, reason enum, description, status (OPEN, UNDER_REVIEW, RESOLVED, DISMISSED), admin_note, resolved_by FK, created_at.

audit_logs: id, actor_id, action, entity, entity_id, metadata JSON, created_at (for admin actions).

settings: key, value (e.g. max_loan_days_default, request_expiry_hours, max_active_loans).

10. REST API Specification

Base URL: `/api/v1`. All responses are JSON. Protected routes need `Authorization: Bearer <token>`. Errors follow `{ "error": { "code": "...", "message": "..." } }`.

Auth

Method	Endpoint	Description
POST	/auth/register	Create account, send verification email
GET	/auth/verify-email?token=	Verify email
POST	/auth/login	Returns access and refresh tokens
POST	/auth/refresh	Rotate access token
POST	/auth/logout	Invalidate refresh token
POST	/auth/forgot-password	Send reset link
POST	/auth/reset-password	Set new password

Users

Method	Endpoint	Description
GET	/users/me	Current profile
PATCH	/users/me	Update profile
GET	/users/:id	Public profile with rating and reviews
DELETE	/users/me	Delete account (blocked if active loans)

Items

Method	Endpoint	Description
GET	/items	List with q, category, condition, available_from, available_to, sort, page, limit
POST	/items	Create item (multipart for images)
GET	/items/:id	Item detail
PATCH	/items/:id	Update (owner)
DELETE	/items/:id	Delete (owner, no active loans)
POST	/items/:id/pause	Toggle pause
GET	/items/:id/availability	Blocked date ranges
GET	/users/me/items	Own listings
GET	/categories	Category list

Loans (Requests)

Method	Endpoint	Description
POST	/items/:id/loans	Borrower creates request
GET	/loans?role=borrower|owner&status=	My loans in either role
GET	/loans/:id	Loan detail
POST	/loans/:id/approve	Owner approves
POST	/loans/:id/reject	Owner rejects with reason
POST	/loans/:id/cancel	Borrower or owner cancels
POST	/loans/:id/handover	Mark handed over
POST	/loans/:id/return	Mark returned
POST	/loans/:id/extension	Request extension
POST	/loans/:id/extension/decision	Owner accepts or declines

Reviews, Notifications, Reports

Method	Endpoint	Description
POST	/loans/:id/reviews	Submit review
GET	/users/:id/reviews	Reviews for a user
GET	/notifications	List mine
PATCH	/notifications/:id/read	Mark read
POST	/notifications/read-all	Mark all read
POST	/reports	File a report

Admin (ADMIN role only)

Method	Endpoint	Description
GET	/admin/stats	Dashboard counts and chart data
GET	/admin/users	Search and filter users
PATCH	/admin/users/:id/status	Warn, suspend, ban, reinstate
GET	/admin/items	All items including removed
PATCH	/admin/items/:id/remove	Take down an item
GET	/admin/loans?status=OVERDUE	Loan monitoring

Method	Endpoint	Description
POST	/admin/loans/:id/remind	Send reminder
GET	/admin/reports	Report queue
PATCH	/admin/reports/:id	Resolve or dismiss
CRUD	/admin/categories	Manage categories
GET/PATCH	/admin/settings	System settings

Example: create a loan request

```
POST /api/v1/items/9f1c.../loans
{
  "startDate": "2026-10-15",
  "endDate": "2026-10-19",
  "message": "Need it for the physics lab on Monday."
}

Response 201:
{ "id": "b3a2...", "status": "PENDING", "itemId": "9f1c...",
  "startDate": "2026-10-15", "endDate": "2026-10-19" }
```

Possible errors: 409 DATES_UNAVAILABLE, 422 EXCEEDS_MAX_DURATION, 403 OWN_ITEM, 403 OVERDUE_LOAN_EXISTS, 429 TOO_MANY_PENDING.

11. Scheduled Jobs

Job	Frequency	Action
Expire pending requests	Every 15 min	Set PENDING older than 48h to EXPIRED, notify borrower
Mark overdue	Hourly	ACTIVE loans with end_date before today become OVERDUE
Due reminders	Daily 8:00	Notify borrowers due tomorrow, today, or overdue; notify the owner of overdue items
Auto-complete	Daily	RETURNED loans older than 7 days become COMPLETED
Start-date nudge	Daily	Remind both parties of approved loans starting today

12. User Interface Specification

Screens

#	Screen	Main elements
1	Landing	Value proposition, search bar, featured categories, sign-up call to action
2	Register / Login	Forms with validation, forgot-password link
3	Home / Browse	Search, filter sidebar (drawer on mobile), item card grid, pagination
4	Item Detail	Image gallery, details, owner card with rating, availability calendar, Request button, reviews, Report link
5	Request Modal	Date range picker, message, summary and rules
6	Add / Edit Item	Form, image uploader with preview, category and condition pickers
7	My Items	Tabs: Listed, Paused; request badges
8	My Loans	Tabs: Borrowing, Lending; status chips and action buttons
9	Loan Detail	Timeline of status, contact info after approval, handover and return buttons, extension request
10	Notifications	List with unread highlighting
11	Profile	Stats, reviews, edit form

#	Screen	Main elements
12	Admin Dashboard	Stat cards, charts, quick links
13	Admin Users / Items / Loans / Reports	Data tables with filters and row actions
14	Admin Settings	Categories and limits

Navigation

- **Member:** Browse, My Loans, My Items, Notifications (bell), Profile.
- **Admin:** adds an Admin section with Dashboard, Users, Items, Loans, Reports, Settings.

UX guidelines

- Mobile-first layout; the primary action (Request / Mark Returned) is always visible.
- Status chips: Pending (amber), Approved (blue), Active (green), Overdue (red), Returned or Completed (grey).
- Empty states with a clear next step; loading skeletons; toast confirmations.
- Confirm dialogs for destructive actions.

13. Security and Privacy

- Hash passwords with bcrypt (cost 12 or higher); never log secrets.
- Short-lived access tokens (15 min) and rotating refresh tokens stored in httpOnly, secure cookies.
- Role-based and ownership checks on every endpoint (e.g. only the owner can approve a loan on their item).
- Validate all input with Zod; use parameterised queries through the ORM to prevent SQL injection.
- Sanitise user-generated text to prevent XSS; set CSP and security headers with Helmet.
- Rate-limit login, register, forgot-password and request-creation endpoints.
- Restrict uploads by type (JPEG, PNG, WebP) and size (5 MB each); strip EXIF metadata.
- Reveal contact details only to the counterparty of an approved loan.
- CORS limited to the known frontend origin; HTTPS enforced.
- Keep an audit log of admin actions; support data export and account deletion.

14. Suggested Project Structure

```
borrowbox/
|-- apps/
|   |-- web/                  # React frontend
|   |   `-- src/
|   |       |-- pages/        # route-level screens
|   |       |-- components/   # ItemCard, LoanTimeline, DateRangePicker...
|   |       |-- features/     # auth, items, loans, admin (hooks + API calls)
|   |       |-- lib/          # api client, auth context, utils
|   |       `-- main.tsx
|   `-- api/                  # Express backend
|       |-- prisma/           # schema.prisma, migrations, seed.ts
|       `-- src/
|           |-- modules/      # auth, users, items, loans, reviews,
|           |   `-- loans/    #   reports, admin, notifications
|           |                 # loans.routes / controller / service / schema .ts
|           |-- middleware/   # auth, rbac, validate, errorHandler, rateLimit
|           |-- jobs/         # cron jobs
|           |-- services/     # email, storage, notification
|           |-- config/       # env, constants
```

```
|           `-- server.ts
|-- packages/shared/            # shared Zod schemas and types
|-- docker-compose.yml          # postgres + api + web
|-- .github/workflows/ci.yml
`-- README.md
```

Conventions: controller handles HTTP, service holds business rules, Prisma accessed only from services. All loan state changes go through one LoanService that enforces the state machine in Section 7.

15. Testing Strategy

Level	Tool	What to cover
Unit	Jest	Date-overlap logic, state transitions, rule checks, rating math
API / integration	Supertest + test DB	Each endpoint, auth and permission failures, concurrent booking attempts
End-to-end	Playwright	Register, list item, request, approve, handover, return, review
Security	OWASP ZAP, npm audit	Common vulnerabilities and dependency issues
Usability	5 to 8 students	Task completion and feedback

Key test cases

1. Request overlapping an approved loan is rejected.
2. Owner cannot request their own item.
3. Non-owner cannot approve or reject.
4. Overdue borrower cannot create new requests.
5. Pending request expires after 48 hours.
6. Review cannot be submitted twice.
7. Suspended user cannot log in.
8. Two simultaneous approvals for overlapping dates: exactly one succeeds.

Quality targets: at least 80% unit coverage on services; all critical flows covered end-to-end.

16. Deployment and DevOps

- **Environments:** local (Docker Compose), staging, production.
- **Config:** .env for DATABASE_URL, JWT_SECRET, JWT_REFRESH_SECRET, SMTP_*, CLOUDINARY_*, APP_URL, ALLOWED_EMAIL_DOMAIN.
- **CI:** on every pull request run lint, type-check, tests and build. On merge to main, deploy to staging; production on tagged release.
- **Hosting (low cost):** frontend on Vercel or Netlify; API and PostgreSQL on Render, Railway or a campus server.
- **Operations:** daily database backups, error tracking (Sentry), uptime checks, structured logs.
- **Migrations:** prisma migrate deploy during release; seed script creates default categories and an admin account.

Seed categories: Books, Calculators, Lab Equipment, Sports Equipment, Electronics, Stationery, Tools, Other.

17. Development Plan

Assuming a team of 3 to 4 students over about 10 weeks.

Sprint	Weeks	Deliverables
0	1	Finalise requirements, wireframes, repo setup, CI, database schema
1	2 to 3	Auth, email verification, profile, role middleware
2	4 to 5	Item CRUD, image upload, categories, browse, search and filters
3	6 to 7	Loan lifecycle, availability calendar, overlap protection, state machine
4	8	Notifications, scheduled reminders, ratings and reviews
5	9	Reports, admin dashboard and moderation tools
6	10	Testing, bug fixes, performance, security pass, deployment, demo and documentation

Suggested work split: one person on backend and database, one on frontend, one on admin and notifications, one on testing, UI and documentation, rotating as needed.

18. Risks and Mitigations

Risk	Mitigation
Items not returned or damaged	Ratings, overdue blocking, admin escalation, condition notes at handover and return
Low initial adoption	Launch with one department, seed popular categories, student club promotion
Fake or abusive accounts	Campus email verification, reporting, admin suspension
Double booking	Database exclusion constraint plus transactional checks
Scope creep	Hold to the MVP list; log extras in the backlog
Privacy concerns	Minimal data, contact shown only after approval, clear privacy policy

19. Future Enhancements

- In-app chat between owner and borrower.
- QR codes on items for quick handover and return check-in.
- Push notifications and a Progressive Web App or native mobile app.
- Department or club shared inventories and library integration.
- Optional deposits or a points and karma system to reward reliable lenders.
- Wishlist and "item wanted" posts.
- Recommendation engine and analytics on campus demand.
- Single sign-on with the campus identity provider.

20. Glossary

Term	Meaning
Lender / Owner	Member who lists an item
Borrower	Member who requests an item
Loan	A request and its full lifecycle
Handover	Physical transfer of the item to the borrower
Overdue	Active loan past its end date
RBAC	Role-based access control

21. Quick-Start Build Checklist

1. Create the monorepo and Docker Compose with PostgreSQL.
2. Define the Prisma schema from Section 9 and run the first migration; add the exclusion constraint through a raw SQL migration.
3. Build auth (register, verify, login, refresh) and RBAC middleware.
4. Build items and categories with image upload.
5. Build the loan service with the state machine, then the request, approve, handover and return endpoints.
6. Add cron jobs and notifications.
7. Build the frontend screens in the order: Auth, Browse, Item Detail, Request, My Loans, My Items.
8. Add reviews, reports and the admin dashboard.
9. Write tests, run a security pass, deploy to staging and run a pilot with a small group of students.